In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

In [3]:
```
data = pd.read_csv("housing.csv")
data.head()
```

Out[3]:

	RM	LSTAT	PTRATIO	MEDV
0	6.575	4.98	15.3	504000.0
1	6.421	9.14	17.8	453600.0
2	7.185	4.03	17.8	728700.0
3	6.998	2.94	18.7	701400.0
4	7.147	5.33	18.7	760200.0

In [4]:
```
print("Shape:", data.shape)
print("\nInfo:")
print(data.info())

print("\nDescription:")
print(data.describe())

print("\nMissing values:")
print(data.isnull().sum())
```

```
Shape: (489, 4)

Info:
<class 'pandas.DataFrame'>
RangeIndex: 489 entries, 0 to 488
Data columns (total 4 columns):
 #   Column   Non-Null Count  Dtype  
---  ------   --------------  -----  
 0   RM       489 non-null    float64
 1   LSTAT    489 non-null    float64
 2   PTRATIO  489 non-null    float64
 3   MEDV     489 non-null    float64
dtypes: float64(4)
memory usage: 15.4 KB
None

Description:
               RM       LSTAT     PTRATIO          MEDV
count  489.000000  489.000000  489.000000  4.890000e+02
mean     6.240288   12.939632   18.516564  4.543429e+05
std      0.643650    7.081990    2.111268  1.653403e+05
min      3.561000    1.980000   12.600000  1.050000e+05
25%      5.880000    7.370000   17.400000  3.507000e+05
50%      6.185000   11.690000   19.100000  4.389000e+05
75%      6.575000   17.120000   20.200000  5.187000e+05
max      8.398000   37.970000   22.000000  1.024800e+06

Missing values:
RM         0
LSTAT      0
PTRATIO    0
MEDV       0
dtype: int64
```

In [5]:
```
X = data.drop("MEDV", axis=1)
y = data["MEDV"]
```

In [6]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

In [7]:
```
model = LinearRegression()
model.fit(X_train, y_train)

print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)
```

```
Intercept: 408027.654168077
Coefficients: [ 87322.20361861 -10620.63731522 -19324.4102965 ]
```

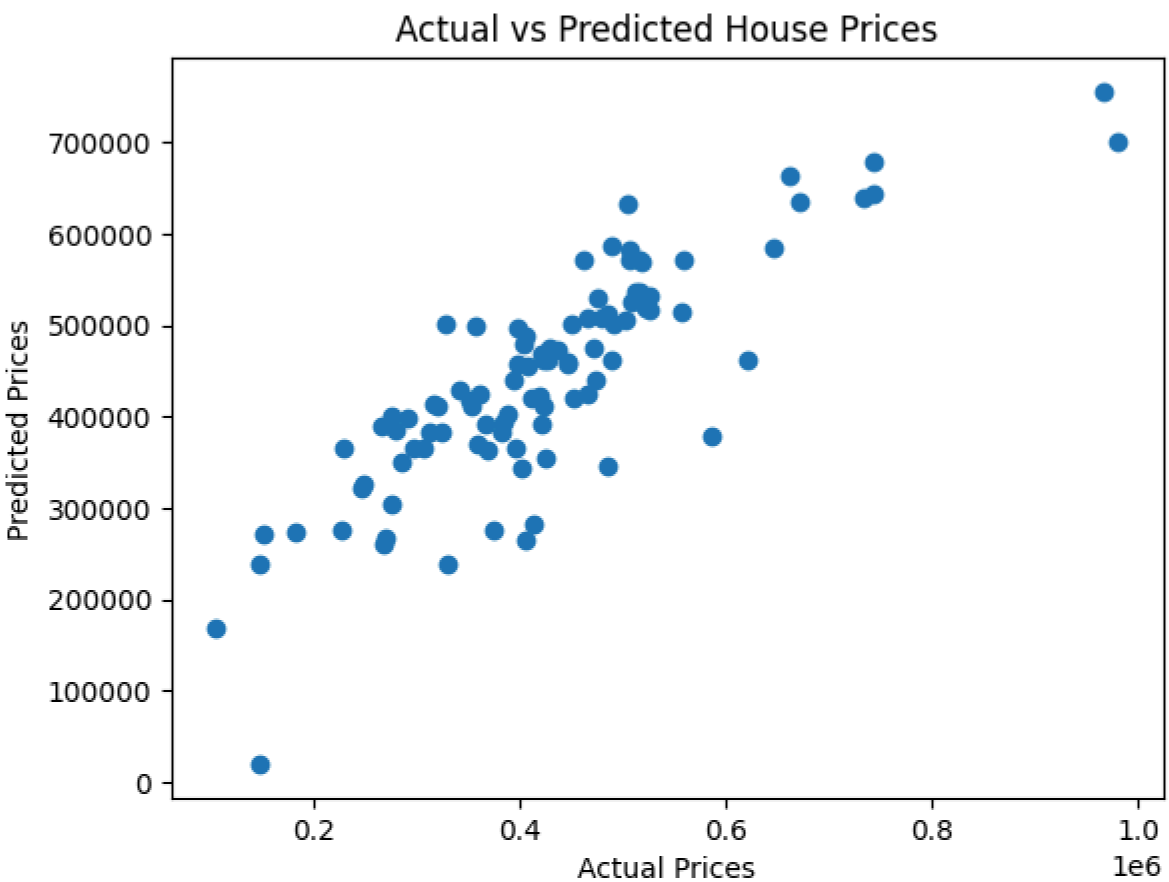
In [8]:
```
y_pred = model.predict(X_test)
print(y_pred[:10])  # first 10 predictions
```

```
[342593.79029768 506257.0916297  410499.93166174 237792.7411537
 327005.79653234 403018.068531   261060.38389067 701308.47374597
 362924.70496746 585818.82333754]
```

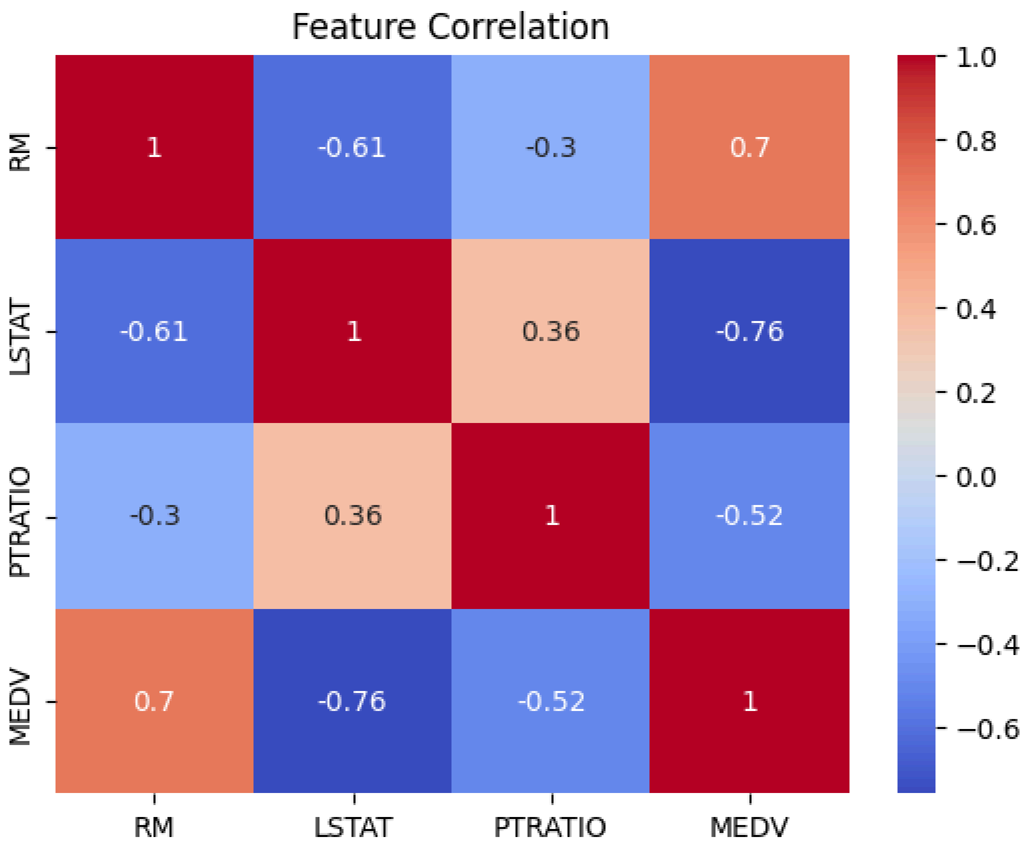
In [9]:
```
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)

r2 = r2_score(y_test, y_pred)
print("R2 Score:", r2)
```

```
Mean Squared Error: 6789025559.265892
R2 Score: 0.691093400309851
```

In [10]:
```
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted House Prices")
plt.show()
```



In [11]:
```
sns.heatmap(data.corr(), annot=True, cmap="coolwarm")
plt.title("Feature Correlation")
plt.show()
```


Feature Correlation
RM
LSTAT
PTRATIO
MEDV
1
-0.61
-0.3
0.7
0.36
-0.76
-0.52
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6

In [ ]: